

Ovire za lamo

Lama želi potovati po Andskem višavju. Na voljo ima zemljevid višavja v obliki mreže kvadratnih celic velikosti $N \times M$. Vrstice zemljevida so oštevilčene od 0 do $N - 1$ od zgoraj navzdol, stolpci pa so oštevilčeni od 0 do $M - 1$ od leve proti desni. Celica zemljevida v vrstici i in stolpcu j ($0 \leq i < N, 0 \leq j < M$) je označena z (i, j) .

Lama je preučila podnebje višavja in ugotovila, da imajo vse celice vsake vrstice zemljevida enako **temperaturo** in vse celice vsakega stolpca enako **vlažnost**. Lama vam je dala dve celoštevilski polji T in H dolžine N oz. M . $T[i]$ ($0 \leq i < N$) označuje temperaturo celic vrstice i , $H[j]$ ($0 \leq j < M$) pa vlažnost celic stolpca j .

Lama je preučila tudi rastje višavja in opazila, da je celica (i, j) **brez rastja**, če je njena temperatura večja od njene vlažnosti, formalno $T[i] > H[j]$.

Lama lahko potuje po višavju samo po **veljavnih poteh**. Veljavna pot je zaporedje različnih celic, ki izpolnjujejo naslednje pogoje:

- Vsak par zaporednih celic na poti ima skupno stranico.
- Vse celice na poti so brez rastja.

Vaša naloga je odgovoriti na Q vprašanj. Vsako vprašanje je sestavljeno iz štirih celih števil: L, R, S , in D . Za vsako vprašanje morate ugotoviti, ali obstaja veljavna pot, kjer:

- Pot se začne v celici $(0, S)$ in konča v celici $(0, D)$.
- Vse celice na poti ležijo znotraj stolpcev L do R .

Poleg tega je zagotovljeno, da sta $(0, S)$ in $(0, D)$ brez rastja.

Podrobnosti implementacije

Najprej morate implementirati proceduro:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : polje dolžine N , ki podaja temperaturo v vsaki vrstici.
- H : polje dolžine M , ki podaja vlažnost v vsakem stolpcu.
- Ta procedura se pokliče natanko enkrat za vsak testni primer, pred katerimkoli klicem `can_reach`.

Nato implementirajte funkcijo:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : cela števila, ki opisujejo vprašanje.
- Funkcija se kliče Q -krat za vsak testni primer.

Ta funkcija mora vrniti `true`, če in samo če obstaja veljavna pot od celice $(0, S)$ do celice $(0, D)$, kjer vse celice poti ležijo znotraj stolpcev L do R , vključno.

Omejitve

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ za vsak i velja $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ za vsak j velja $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Celici $(0, S)$ in $(0, D)$ sta brez rastja.

Podnaloge

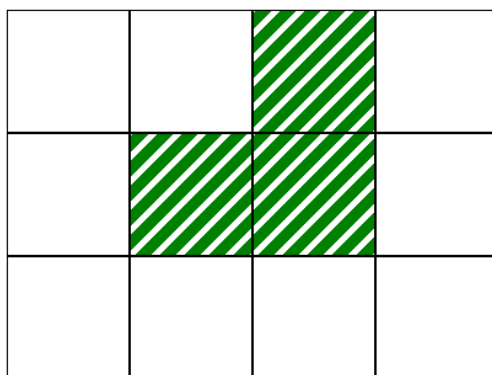
Podnaloga	Točke	Dodatne omejitve
1	10	$L = 0, R = M - 1$ za vsako vprašanje. $N = 1$.
2	14	$L = 0, R = M - 1$ za vsako vprašanje. $T[i - 1] \leq T[i]$ za vsak i , kjer $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ za vsako vprašanje. $N = 3, T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ za vsako vprašanje. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ za vsako vprašanje.
6	17	Ni dodatnih omejitev.

Primer

Razmislite o naslednjem klicu:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

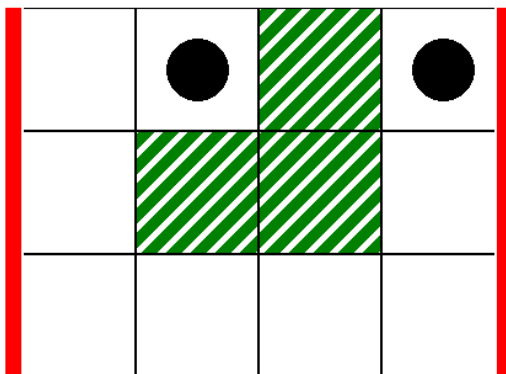
To ustreza zemljevidu na naslednji sliki, kjer so bele celice brez rastja:



Kot prvo vprašanje razmislite o naslednjem klicu:

```
can_reach(0, 3, 1, 3)
```

To ustreza scenariju na naslednji sliki, kjer debele navpične črte označujejo obseg stolpcev od $L = 0$ do $R = 3$, črne pike pa začetno in končno celico:



V tem primeru lahko lama pride iz celice $(0, 1)$ do celice $(0, 3)$ po naslednji veljavni poti:

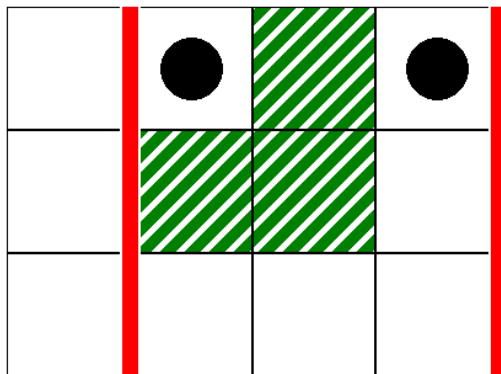
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Zato mora ta klic vrniti `true`.

Kot drugo vprašanje razmislite o naslednjem klicu:

```
can_reach(1, 3, 1, 3)
```

To ustreza scenariju na naslednji sliki:



V tem primeru ni veljavne poti od celice $(0,1)$ do celice $(0,3)$, kjer bi vse celice poti ležale znotraj stolpcev 1 do 3, vključno. Zato mora ta klic vrniti `false`.

Vzorčni ocenjevalnik

Oblika vhoda:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Tu $L[k]$, $R[k]$, $S[k]$ in $D[k]$ ($0 \leq k < Q$) označujejo parametre za vsak klic `can_reach`.

Oblika izhoda:

```
A[0]
A[1]
...
A[Q-1]
```

Tu $A[k]$ ($0 \leq k < Q$) je 1, če klic `can_reach(L[k], R[k], S[k], D[k])` vrne `true`, in 0 v nasprotnem primeru.